KUBERNETES WORKLOAD SPECIFICATION & RUNBOOK

Accelerated PyTorch Workload Deployment

Workload: **gdx-app-pod** Service: **gdx-app-service (NodePort)** Framework: **NVIDIA PyTorch NGC**

1. Architecture Overview

This operational specification details the architecture and deployment procedure for a containerized deep learning environment. The workload provisions an accelerated PyTorch environment featuring dual network services: an interactive JupyterLab notebook environment running as an asynchronous daemon, and an auxiliary Python HTTP server providing lightweight file inspection. External ingress is routed through a Kubernetes NodePort Service.

Configuration Field	Manifest Value	Architectural Purpose
Base Image	`nvcr.io/nvidia/pytorch:26.10-py3`	NVIDIA NGC optimized container with PyTorch, CUDA, and cuDNN pre-installed.
QoS Class	Guaranteed (Requests == Limits)	4 CPU cores, 18 GiB RAM, and 1 dedicated `nvidia.com/gpu` allocation.
Shared Memory	`/dev/shm` (18 GiB `emptyDir`)	Prevents PyTorch DataLoader IPC worker deadlocks and `SIGBUS` failures.
Security Capabilities	`SYS_PTRACE`	Authorizes process inspection and profiling tools (`nsys`, `cuda-gdb`).
Network Service	`NodePort` on port 8888	Exposes JupyterLab externally on each cluster node's IP.

2. Complete Kubernetes Manifest

Save the following configuration as `gdx-app.yaml` in your deployment repository:

```
apiVersion: v1
kind: Pod
metadata:
  name: gdx-app-pod
  labels:
    app: gdx-app
spec:
  containers:
    - name: gdx-container
      image: nvcr.io/nvidia/pytorch:26.10-py3
      imagePullPolicy: IfNotPresent
      command: ["/bin/bash", "-c"]
      args:
        - |
          jupyter lab --ip=0.0.0.0 --port=8888 --no-browser --allow-root &
          python3 -m http.server 8080
      ports:
        - name: jupyter-port
          containerPort: 8888
          protocol: TCP
        - name: api-port
          containerPort: 8080
          protocol: TCP
      resources:
        requests:
          memory: "18Gi"
          cpu: "4"
          nvidia.com/gpu: "1"
        limits:
          memory: "18Gi"
          cpu: "4"
          nvidia.com/gpu: "1"
      securityContext:
        capabilities:
          add: ["SYS_PTRACE"]
      volumeMounts:
        - name: dshm
          mountPath: /dev/shm
  volumes:
    - name: dshm
      emptyDir:
        medium: Memory
        sizeLimit: 18Gi
---
apiVersion: v1
kind: Service
metadata:
  name: gdx-app-service
  labels:
    app: gdx-app
spec:
  type: NodePort
  selector:
    app: gdx-app
  ports:
```

3. Technical Design Considerations

Shared Memory Mount (`/dev/shm`)

By default, standard container runtimes (such as containerd and CRI-O) allocate a restrictive 64 MiB shared memory partition to containers. PyTorch multi-processing data loaders (`torch.utils.data.DataLoader` with `num_workers > 0`) transfer tensors between worker processes via POSIX shared memory. Under deep learning workloads, exceeding 64 MiB results in fatal `SIGBUS` (bus error) crashes. This configuration provisions a dedicated RAM-backed `emptyDir` volume sized at 18 GiB mounted to `/dev/shm` .

Security Capabilities (`SYS_PTRACE`)

Deep learning performance optimization requires profiling memory bandwidth, kernel execution times, and compute efficiency. Enabling `SYS_PTRACE` provides the necessary kernel privileges inside the container for debugging utilities such as NVIDIA Nsight Systems (`nsys`), `cuda-gdb` , and Python profilers (e.g., `py-spy`) without granting full privileged container status.

> **Guaranteed Quality of Service (QoS):** Specifying identical CPU and Memory requests and limits ensures the Pod receives Kubernetes *Guaranteed* QoS, minimizing node eviction risk during intensive model training cycles.

4. Step-by-Step Deployment Guide

Step 1: Verify Cluster Prerequisites

Ensure that the NVIDIA Container Toolkit / GPU Operator is healthy and that nodes report available GPU capacity:

```
kubectl get nodes "-o=custom-
columns=NAME:.metadata.name,GPU:.status.allocatable.nvidia\.com/gpu"
```

Step 2: Deploy the Workload

Apply the manifest directly to the desired namespace:

```
kubectl apply -f gdx-app.yaml
```

Step 3: Monitor Pod Initialization

NVIDIA NGC images contain extensive CUDA and deep learning toolsets. Initial layer pulling may take several minutes:

```
kubectl get pod gdx-app-pod -w
kubectl describe pod gdx-app-pod
```

Step 4: Extract JupyterLab Access Token

Once the container state transitions to `Running`, retrieve the initialization token from stdout logs:

```
kubectl logs gdx-app-pod -c gdx-container | grep -E "token=[a-f0-9]+"
```

Step 5: Access JupyterLab via NodePort

Locate the assigned high-range node port and node network addresses:

```
kubectl get svc gdx-app-service -o jsonpath='{.spec.ports[0].nodePort}'
kubectl get nodes -o wide
```

Navigate to: `http://<NODE_IP>:<NODE_PORT>/lab?token=<TOKEN>`

Step 6: Access Auxiliary HTTP Server (Port 8080)

Because the NodePort Service deliberately targets only port 8888, access port 8080 via port-forwarding:

```
kubectl port-forward pod/gdx-app-pod 8080:8080
```

5. Verification & Sanity Checks

Execute an interactive session inside the container to validate hardware acceleration and shared memory allocation:

```
# Open container shell
kubectl exec -it gdx-app-pod -- /bin/bash

# 1. Verify GPU visibility
nvidia-smi

# 2. Confirm /dev/shm size allocation
df -h /dev/shm

# 3. Test CUDA availability in PyTorch
python3 -c "import torch; print('CUDA Ready:', torch.cuda.is_available());
print('Device:', torch.cuda.get_device_name(0))"
```